M V P C O M P A N I O N · RESOURCE 1.2

Stop Building Features Before Answering These Questions

A practical guide for founders and developers who want to build the right MVP—not just a longer feature list.

Before you decide what to build, clarify the problem, the user, the outcome and what you need to learn.

Feature-first thinking feels productive. You can list screens, integrations and AI capabilities quickly. But a feature list does not establish that a real problem exists, that a particular user cares about it, or that the proposed product will deliver meaningful value.

MVPCompanion uses progressive clarification: move from an uncertain idea toward a focused, testable and buildable MVP specification. The aim is not to remove every uncertainty before starting. It is to make important assumptions visible and ensure that product decisions have a clear rationale.

Seven questions to answer before expanding your feature list

1. Are you describing a problem—or already prescribing a solution?

Weak: “Users need an AI-powered dashboard.” This names a solution. Ask instead: Who is struggling, in what situation, what are they trying to accomplish, and what makes the current approach difficult? A problem should be understandable without naming your proposed app or its features.

2. Who has this problem most directly?

“Everyone” is not a useful initial target. Identify a primary user, their context, constraints, current workflow and alternatives. State who is not the initial target, too. A clear target helps you judge which Jobs and capabilities matter most.

3. What outcome does the user actually want?

Describe the progress or result the user wants—not the screen they will click or the technology you plan to use. Your value proposition should make sense without a feature list. Ask why the outcome matters and what users do today instead.

4. What is evidence, and what is still an assumption?

Do not let a plausible idea quietly become a “fact.” Label statements as Evidence (E), Hypothesis (H), Derived (D) or Open (O). Record the basis for evidence, show what a conclusion is derived from, and keep unresolved questions visible.

5. What must the user accomplish? (Core Jobs)

A Job describes what a user is trying to accomplish, not a UI element or feature. Identify and prioritize the Core Jobs that support the promised outcome. For each, note the trigger or situation, the desired result and the minimum information or decision needed.

Seven questions, continued

6. Does every proposed feature support a Core Job?

Derive capabilities from prioritized Core Jobs. For each feature, ask which Job it supports, what happens if you remove it, whether it is needed for the end-to-end journey, and what evidence supports including it now. If the rationale is unclear, challenge the feature or record the open question.

7. What must the MVP teach you?

An MVP is not only software that works; it should help test important assumptions. Identify the learning question, the observable signal that may inform it, and what result could change your decision. Choose measures that connect to the product's intended value rather than tracking activity simply because it is easy.

A fast feature challenge

Before accepting a feature into the MVP, complete this mini-review:

Question	Decision prompt
Job link	Which prioritized Core Job does this support?
Outcome	How does it help the user achieve the intended outcome?
Necessity	What breaks if we remove it? Is the core journey still coherent?
Evidence	What supports the decision to include it now? What remains a hypothesis?
Priority	Must Have, Should Have, Later, or Not in MVP? Why?
Learning	Does it test an important assumption or produce a meaningful signal?

Use explicit scope categories

- **Must Have:** needed for the smallest coherent product to deliver its core promise or test a critical assumption.
- **Should Have:** valuable, but not essential to the initial coherent MVP.
- **Later:** potentially useful after more learning or development.
- **Not in MVP:** explicitly excluded from the current boundary.

The goal is not the fewest features. It is the smallest coherent solution that supports the core Jobs, delivers the intended value and enables meaningful learning.

Keep the reasoning connected

Use MVPCompanion's traceability chain to check whether a feature has a reason to exist:

Problem → User → Outcome → Job → Capability → Requirement → Acceptance Criterion → Metric

If a capability cannot be traced back to a meaningful user outcome, challenge it. If an important outcome has no supporting capability, the MVP may be incomplete. Make exclusions explicit and revisit decisions when meaningful new evidence appears.

AI can help ask questions, expose contradictions and suggest alternatives. It does not replace human judgment about which problem matters, what evidence is sufficient or which trade-offs to accept.

Your next step: Start with an Idea Brief, clarify the problem and primary user, then prioritize Core Jobs before deriving capabilities. Use the MVP Scope Challenge to test each proposed feature.

Aligned with MVPCompanion Method V2.0 · 15 Stages of Progressive Clarification. This short guide is an orientation resource, not a substitute for the full method specification. Examples are illustrative and do not constitute market evidence.